



Université Cadi Ayyad
École Supérieure de Technologie
Essaouira
Filière : Informatique
Décisionnelle
et Science des Données



RAPPORT DE STAGE DE FIN D'ÉTUDES

Pour l'obtention du Diplôme Universitaire de Technologie (DUT)

Conception et Développement d'un Assistant intelligent pour SQL

Présenté par :
Abahous Nor-eddine

Encadrant pédagogique :
M. Ait Ouakrim Abdessamad

Année Universitaire : 2026 – 2027

Remerciements

Je tiens avant tout à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage et à la rédaction de ce rapport.

Mes remerciements les plus sincères vont à **M. Ait Ouakrim Abdessamad**, mon encadrant pour son disponibilité, ses conseils avisés et son suivi rigoureux tout au long de cette période. Ses orientations ont été déterminantes dans la structuration du projet et dans la qualité du travail accompli.

Je remercie également l'ensemble du corps professoral de la filière *Informatique Décisionnelle et Science des Données* pour la formation solide dispensée, qui m'a permis d'aborder ce projet avec les outils conceptuels et techniques nécessaires.

Mes vifs remerciements s'adressent aussi à la direction et aux équipes techniques de **COPAG** pour m'avoir accueilli dans leurs locaux, facilité l'accès aux données et systèmes nécessaires, et fait confiance à un étudiant stagiaire pour un projet à réelle valeur ajoutée.

Enfin, je remercie ma famille et mes proches pour leur soutien constant, leur patience et leurs encouragements tout au long de cette aventure académique et professionnelle.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage de fin d'études effectué au sein de la coopérative **COPAG** (Coopératives Bladna), en vue de l'obtention du Diplôme Universitaire de Technologie en Informatique Décisionnelle et Science des Données.

Le projet s'inscrit dans une démarche de modernisation du système décisionnel de COPAG. Face à des données commerciales volumineuses et dispersées entre différentes sources hétérogènes, l'entreprise cherchait à démocratiser l'accès à l'information analytique pour ses équipes métier, sans exiger de compétences SQL.

La solution développée repose sur une architecture en trois couches : un **Data Lake** qui centralise les données brutes provenant des systèmes sources (ERP, fichiers CSV), un **Data Warehouse Snowflake** qui structure ces données selon un schéma en étoile optimisé pour l'analytique, et une **interface Text-to-SQL** alimentée par le moteur *Vanna.ai* et le LLM *Groq (Llama 3)*, permettant à tout utilisateur de poser des questions en langage naturel et d'obtenir instantanément des visualisations et des analyses.

Abstract

This report presents work carried out during a final-year internship at COPAG (Coopératives Bladna), in partial fulfillment of a University Diploma of Technology in Decision Support Systems and Data Science.

The project addresses the challenge of making COPAG's commercial data accessible to business users without SQL expertise. The solution implements a three-layer data architecture : a Data Lake consolidating raw data from heterogeneous sources, a Snowflake Data Warehouse structured as a star schema, and a Text-to-SQL interface powered by Vanna.ai and the Groq LLM, enabling natural language queries with automatic chart generation.

Table des matières

Remerciements	i
Résumé	ii
Table des figures	v
Liste des tableaux	vi
1 Présentation de la Société COPAG	2
1.1 Introduction	2
1.2 Historique	2
1.3 Fiche Technique	3
1.4 Missions	3
1.5 Conclusion	4
2 Contexte Général et Enjeux	6
2.1 Introduction	6
2.2 Problématique	6
2.3 Cycle de Vie du Projet	7
2.3.1 Objectifs du Projet	7
2.3.2 Conception	7
2.3.3 Phase de Réalisation	8
2.3.4 Phase de Test	8
2.4 Conclusion	9
3 Analyse et Conception	11
3.1 Introduction	11
3.2 Diagrammes et Schémas	11
3.2.1 Schéma en Flocon (Snowflake Schema)	11
3.2.2 Schéma du Flux de Données	13
3.3 Conclusion	15

4 Étude de la Partie Logicielle et Outils de Travail	17
4.1 Introduction	17
4.2 Outils et Technologies Utilisés	17
4.3 Conclusion	19
4.4 Introduction	21
4.5 Création et Gestion des Bases de Données	21
4.6 Extraction et Chargement des Données Sources vers le Data Lake	21
4.7 Automatisation et Synchronisation	23
4.8 Migration et Transformation des Données	23
4.9 Entraînement de Vanna avec 35 Exemples et Schéma du Data Warehouse .	24
4.10 Création de l'Interface et Tests	25
4.10.1 Partie des Questions	25
4.10.2 Partie de Visualisation	26
4.11 Conclusion	27
Conclusion Générale	29
Webographie	30

Table des figures

1.1	Logo officiel de COPAG — Coopératives Bladna	3
2.1	Architecture globale non détaillée du système décisionnel	8
3.1	Schéma en flocon (Snowflake) du Data Warehouse COPAG	12
3.2	Flux de données détaillé — du système source au Data Warehouse	13
3.3	Diagramme de séquence — Processus de génération et d'exécution d'une requête SQL	14
4.1	Pipeline SSIS — Migration des données sources vers le Data Lake	21
4.2	Processus détaillé de migration source → Data Lake	22
4.3	Processus d'automatisation — Détail du flux de données article (CDC Source et CDC Splitter)	22
4.4	Package SSIS d'automatisation et synchronisation incrémentale (INI_SYN)	23
4.5	Pipeline SSIS — Migration Data Lake vers Data Warehouse (dimensions)	23
4.6	Processus de chargement Data Lake → Data Warehouse (DIM_PRODUT, SCD Type 1)	24
4.7	Page Principal	26
4.8	Page de résultats — Après soumission d'une requête	26
4.9	Page Visualisation Analytique	27

Liste des tableaux

1.1	Fiche technique de COPAG	3
3.1	Tables du Data Warehouse COPAG	12
4.1	Récapitulatif des technologies utilisées par couche	17
4.2	Exemples de questions entraînées sur Vanna (extrait)	25

Chapitre 1

Présentation de la Société COPAG

Chapitre 1

Présentation de la Société COPAG

1.1 Introduction

Ce chapitre présente la société COPAG, son historique, sa fiche technique et ses missions. Cette présentation constitue le socle contextuel indispensable pour comprendre les enjeux du projet réalisé pendant ce stage.

1.2 Historique

La coopérative COPAG a été fondée le 7 mai 1987, lors d'une assemblée générale réunissant 39 agriculteurs de la région de Taroudant. Ces agriculteurs, voulant reprendre le contrôle de la filière agricole locale, ont saisi l'opportunité offerte par la politique de libéralisation des exportations initiée par l'État marocain à cette période.

Depuis ses débuts, COPAG s'est imposée comme l'une des plus belles réussites coopératives du Maroc. Son modèle — fondé sur la solidarité des membres et une vision stratégique partagée — lui a permis de se développer sur tous les maillons de la chaîne de valeur : de la production agricole jusqu'à la commercialisation des produits finis.

Aujourd'hui, COPAG est un acteur de référence dans l'industrie agro-alimentaire marocaine. La coopérative commercialise ses produits sous la marque **Bladna**, reconnue dans tout le Royaume pour ses produits laitiers, ses jus de fruits et ses dérivés agricoles.



FIGURE 1.1 – Logo officiel de COPAG — Coopératives Bladna

1.3 Fiche Technique

TABLE 1.1 – Fiche technique de COPAG

Critère	Informations
Raison sociale	COPAG — Coopératives Agricoles
Marque commerciale	Bladna
Date de création	7 mai 1987
Siège social	Taroudant, Maroc
Secteur d'activité	Agro-alimentaire / Coopérative agricole
Domaines	Produits laitiers, jus de fruits, dérivés agricoles
Zone de couverture	Nationale (tout le Maroc)
Statut juridique	Coopérative agricole
Effectif	Plusieurs centaines de salariés

1.4 Missions

COPAG poursuit trois grandes missions :

1. **Développement socio-économique rural** : assurer, directement ou par le biais de ses adhérents, le développement économique et social du milieu rural de la région du Souss.
2. **Valorisation des produits agricoles** : offrir des produits d'origine animale et végétale de plus en plus élaborés, répondant aux attentes actuelles des consommateurs en matière de qualité, de sécurité alimentaire et de diversité.
3. **Amélioration des revenus** : accroître le revenu de la coopérative et de ses adhérents

grâce à une chaîne de valeur maîtrisée, de la production jusqu'à la commercialisation, en passant par la transformation.

1.5 Conclusion

Ce chapitre a présenté COPAG, coopérative agricole marocaine fondée en 1987, devenue un acteur structurant du secteur agro-alimentaire national. Sa trajectoire illustre ce que le modèle coopératif peut produire quand il est porté par une vision claire. C'est dans ce contexte que s'inscrit le projet de modernisation décisionnelle objet de ce stage, dont les chapitres suivants décrivent les enjeux et les réalisations.

Chapitre 2

Contexte Général et Enjeux

Chapitre 2

Contexte Général et Enjeux

2.1 Introduction

Ce chapitre pose le cadre général du projet de stage. Il décrit la problématique à laquelle COPAG est confrontée, les objectifs fixés, le cycle de vie retenu pour conduire le développement, et les grandes phases de réalisation et de test.

2.2 Problématique

COPAG génère quotidiennement des volumes importants de données commerciales : ventes par article, par agence, par secteur, retours clients, invendus, données promotionnelles, etc. Ces données sont stockées dans des systèmes sources hétérogènes — un ERP métier, des fichiers CSV exportés manuellement, des bases opérationnelles — rendant leur exploitation analytique difficile.

Deux problèmes principaux se posent :

- **Silotage des données** : les informations sont dispersées, sans couche d'intégration unifiée. Produire un rapport consolidé exige des manipulations manuelles longues et sujettes aux erreurs.
- **Barrière technique à l'accès aux données** : seules les personnes maîtrisant SQL peuvent interroger directement les bases. Les équipes commerciales et la direction dépendent des équipes IT pour obtenir des chiffres, ce qui ralentit la prise de décision.

L'objectif du stage est de répondre à ces deux problèmes en construisant :

1. une architecture de données centralisée (Data Lake → Data Warehouse),
2. une interface en langage naturel permettant à tout utilisateur d'interroger ces données sans écrire une ligne de SQL.

2.3 Cycle de Vie du Projet

2.3.1 Objectifs du Projet

Les objectifs opérationnels du projet sont :

- Mettre en place un **Data Lake** sur SQL Server pour centraliser les données brutes issues de toutes les sources.
- Concevoir un **Data Warehouse** (Snowflake schema) optimisé pour les requêtes analytiques.
- Développer des pipelines **ETL/CDC** avec SSIS pour automatiser l'extraction, la transformation et le chargement des données.
- Implémenter un assistant **Text-to-SQL** basé sur Vanna.ai, capable de générer des requêtes SQL à partir de questions en français.
- Créer une **interface web** avec Streamlit permettant de visualiser les résultats sous forme de tableaux et de graphiques interactifs.

2.3.2 Conception

La phase de conception a couvert :

- l'identification des sources de données et de leur structure,
- la modélisation du schéma en flocon (*Snowflake schema*) du Data Warehouse,
- la définition de l'architecture globale (Data Lake, ETL, Data Warehouse, couche applicative),
- la conception des flux de données et des diagrammes de séquence.

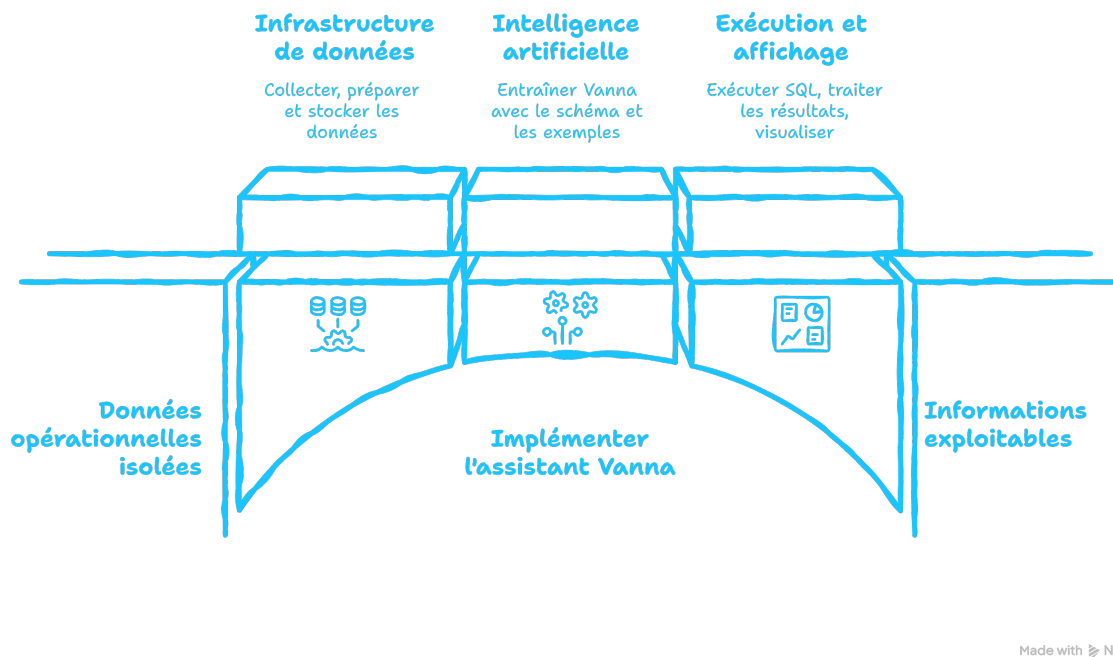


FIGURE 2.1 – Architecture globale non détaillée du système décisionnel

2.3.3 Phase de Réalisation

La réalisation a suivi un ordre logique :

1. Création des bases de données (Data Lake et Data Warehouse) dans SQL Server.
2. Développement des packages SSIS pour la migration source → Data Lake.
3. Développement des packages SSIS pour la migration Data Lake → Data Warehouse.
4. Automatisation via CDC (*Change Data Capture*).
5. Intégration de Vanna.ai et entraînement sur le schéma COPAG.
6. Développement de l'interface Streamlit.

2.3.4 Phase de Test

Les tests ont porté sur :

- la vérification de l'intégrité des données après chaque migration,
- la qualité des requêtes SQL générées par Vanna à partir de 35 questions de test,
- la pertinence des visualisations produites automatiquement,
- les performances de l'interface sous charge.

2.4 Conclusion

Ce chapitre a établi le contexte du projet et ses objectifs. La problématique de silotage des données et d'inaccessibilité de l'information analytique justifie pleinement la mise en place d'une architecture décisionnelle moderne, dont la conception est détaillée au chapitre suivant.

Chapitre 3

Analyse et Conception

Chapitre 3

Analyse et Conception

3.1 Introduction

Ce chapitre présente l'architecture de la solution à travers les principaux artefacts de conception : le schéma en flocon du Data Warehouse, les flux de données (data flows), et le diagramme de séquence de l'interface Text-to-SQL.

3.2 Diagrammes et Schémas

3.2.1 Schéma en Flocon (Snowflake Schema)

Le Data Warehouse adopte un **schéma en flocon** (*Snowflake Schema*), variante normalisée du schéma en étoile. Ce choix est motivé par la hiérarchie produit de COPAG, qui s'organise naturellement en :

$$\text{GAMME} \rightarrow \text{FAMILLE} \rightarrow \text{SOUS_FAMILLE} \rightarrow \text{ARTICLE}$$

Et la hiérarchie géographique :

$$\text{AGENCE} \rightarrow \text{SECTEUR}$$

La table de faits centrale **FACT_SALES** enregistre chaque transaction commerciale avec les métriques quantitatives : quantité vendue (**qty_unite**), prix de vente (**prix_vente**), retours (**qty_rendu**), invendus (**qty_invendu**), et ventes promotionnelles (**qty_promo**).

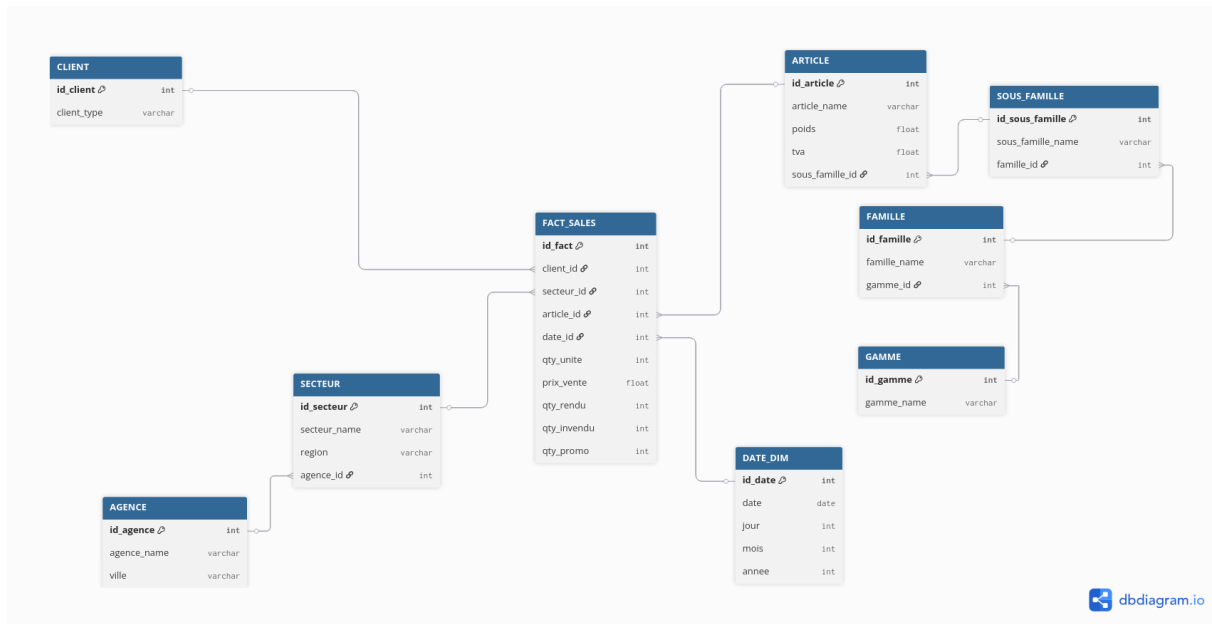


FIGURE 3.1 – Schéma en flocon (Snowflake) du Data Warehouse COPAG

Le schéma comprend les tables de dimensions suivantes :

TABLE 3.1 – Tables du Data Warehouse COPAG

Table	Type	Description
FACT_SALES	Faits	Transactions de vente avec métriques
CLIENT	Dimension	Canal, nom et code client
AGENCE	Dimension	Agence commerciale et ville
SECTEUR	Dimension	Secteur commercial et région
GAMME	Dimension	Gamme de produits
FAMILLE	Dimension	Famille de produits
SOUS_FAMILLE	Dimension	Sous-famille de produits
ARTICLE	Dimension	Article avec poids, TVA et prix de référence
DATE_DIM	Dimension	Axe temporel (jour, mois, année)

3.2.2 Schéma du Flux de Données

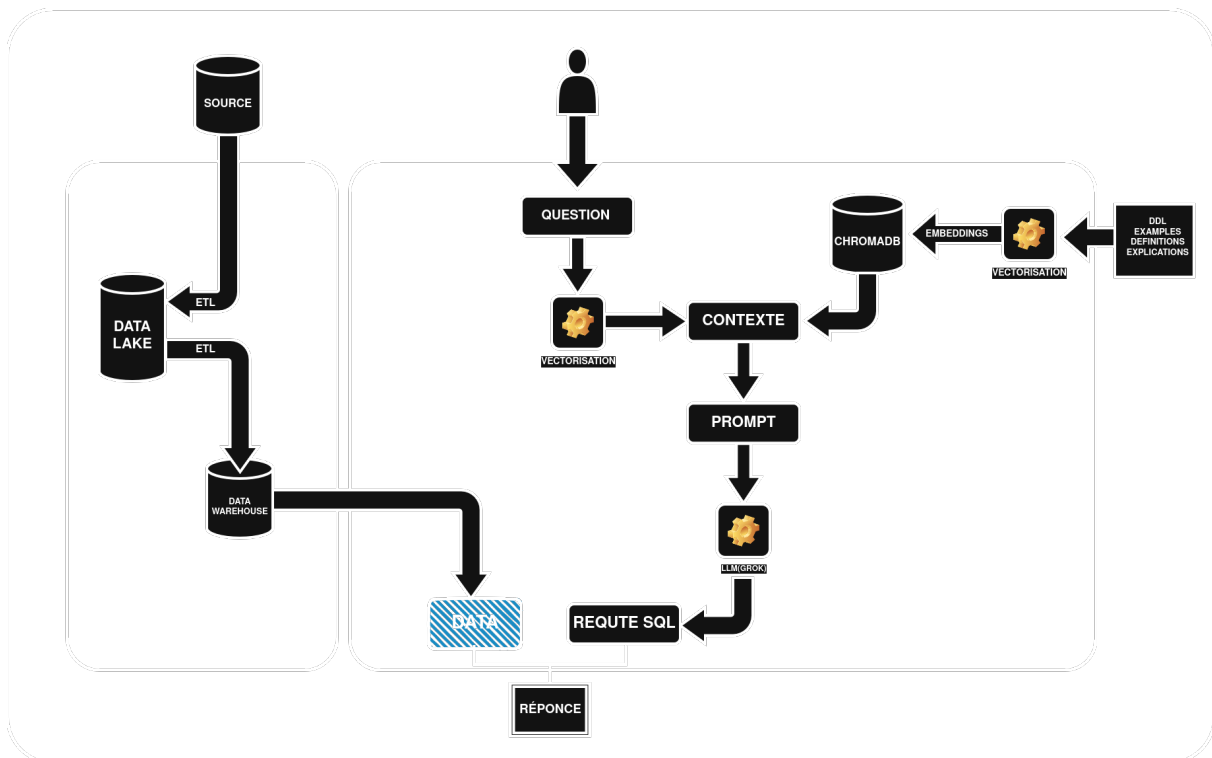


FIGURE 3.2 – Flux de données détaillé — du système source au Data Warehouse

Le flux de données se décompose en trois grandes étapes :

1. **Extraction depuis les sources** : les données brutes provenant de l'ERP et des fichiers CSV sont extraites via des connexions OLE DB dans SSIS.
2. **Stockage dans le Data Lake** : les données brutes sont chargées telles quelles dans le Data Lake SQL Server, sans transformation, pour conserver la traçabilité.
3. **Transformation et chargement dans le Data Warehouse** : les données du Data Lake sont nettoyées, transformées et structurées selon le schéma en flocon.

Diagramme de séquence — Interface Text-to-SQL

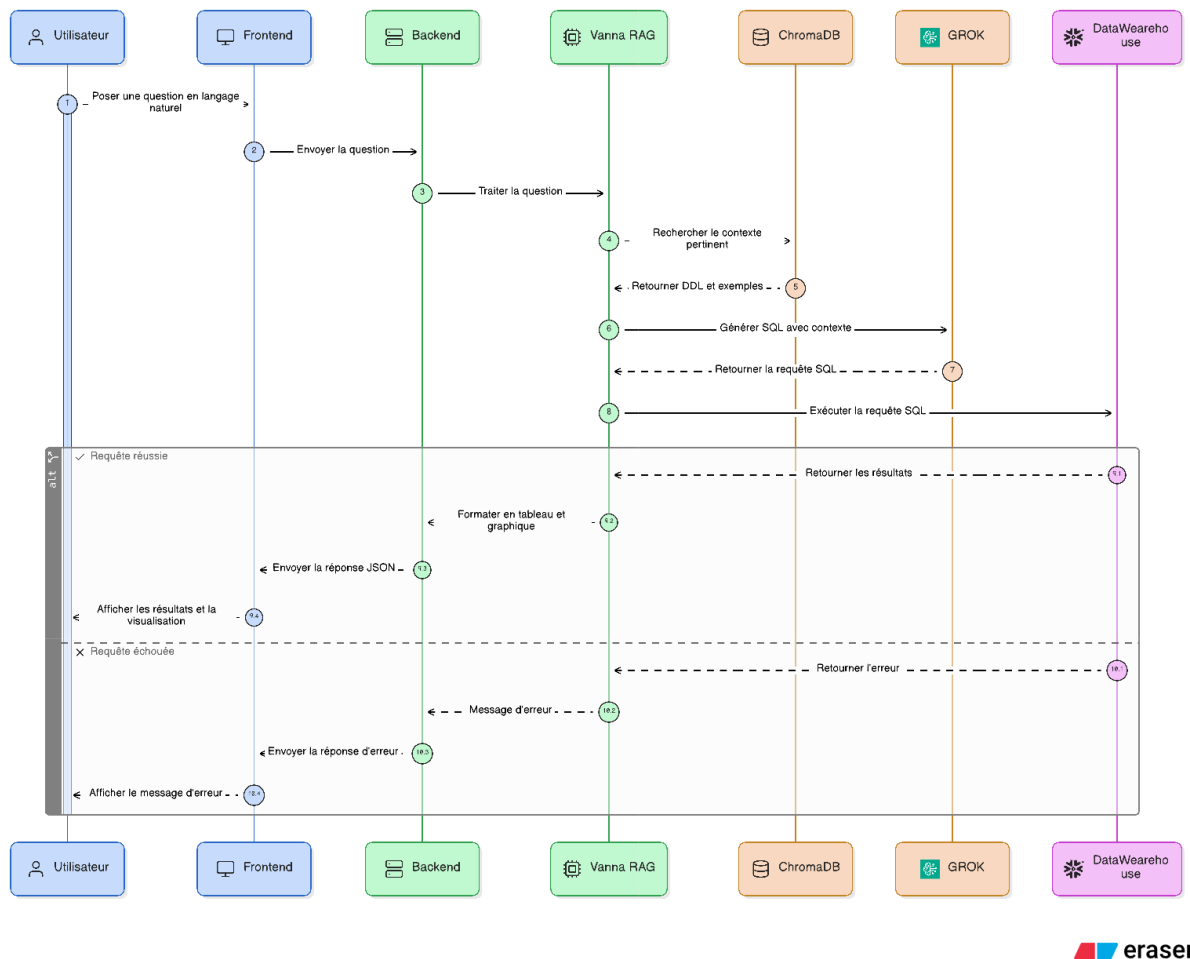


FIGURE 3.3 – Diagramme de séquence — Processus de génération et d’exécution d’une requête SQL

Le diagramme de séquence illustre les interactions entre les composants lors d’une requête :

1. L’utilisateur saisit une question en langage naturel dans l’interface.
2. Le **Backend** transmet la question à **Vanna RAG**.
3. Vanna interroge **ChromaDB** pour récupérer le contexte pertinent (DDL, exemples SQL).
4. Vanna appelle **GROK (LLM)** pour générer la requête SQL avec ce contexte.
5. Le SQL généré est exécuté sur le **Data Warehouse**.
6. Les résultats sont formatés et renvoyés à l’interface pour affichage.

3.3 Conclusion

Ce chapitre a posé les bases architecturales du projet. Le schéma en flocon répond précisément à la hiérarchie produit et géographique de COPAG. Le diagramme de séquence confirme la faisabilité technique du pipeline Text-to-SQL. Le chapitre suivant détaille les outils et technologies retenus pour l'implémentation.

Chapitre 4

Étude de la Partie Logicielle
et Outils de Travail

Chapitre 4

Étude de la Partie Logicielle et Outils de Travail

4.1 Introduction

Ce chapitre présente l’environnement technique du projet — les outils, les frameworks et les technologies retenus pour chaque couche de l’architecture. Le choix de chaque outil est justifié par les contraintes fonctionnelles et les besoins spécifiques du projet COPAG.

4.2 Outils et Technologies Utilisés

TABLE 4.1 – Récapitulatif des technologies utilisées par couche

Couche	Outil / Technologie	Rôle
Stockage	SQL Server	Hébergement du Data Lake et des tables sources
Stockage	Snowflake (schéma)	Modélisation du Data Warehouse analytique
ETL	SSIS	Pipelines d’extraction, transformation, chargement
ETL	CDC	Capture des changements de données en temps réel
IA / LLM	Vanna.ai	Framework Text-to-SQL avec RAG
IA / LLM	Groq (Llama 3)	Génération de SQL à partir de contexte vectoriel
Vectoriel	ChromaDB	Base de données vectorielle locale pour le RAG
Interface	Streamlit	Application web interactive en Python
Visualisation	Plotly	Graphiques interactifs (barres, courbes, donut)
Langage	Python 3.10+	Langage principal de développement applicatif

SQL Server Integration Services (SSIS)

SSIS est le moteur ETL de Microsoft SQL Server. Il offre une interface graphique de conception de flux de données, ce qui accélère le développement des pipelines. Les deux fonctionnalités les plus utilisées dans ce projet sont :

- **Data Flow Task** : transport des données d'une source OLE DB vers une destination OLE DB.
- **CDC (Change Data Capture)** : suivi des modifications (INSERT, UPDATE, DELETE) dans les tables sources, permettant une synchronisation incrémentale sans rechargement complet.

Vanna.ai

Vanna.ai est un framework open-source Python spécialisé dans la génération de SQL en langage naturel. Il repose sur une architecture **RAG** (*Retrieval Augmented Generation*) : avant de générer une requête, il récupère dans ChromaDB les définitions DDL, les exemples de requêtes et le glossaire métier les plus pertinents, puis les injecte dans le contexte du LLM.

Groq / Llama 3

Groq est une plateforme d'inférence LLM ultra-rapide, compatible avec l'API OpenAI. Le modèle *llama-3.3-70b-versatile* a été retenu pour sa capacité à générer du SQL complexe (jointures multiples, agrégations, fenêtres) en moins d'une seconde.

ChromaDB

ChromaDB est une base de données vectorielle locale. Elle stocke les embeddings des DDL, exemples SQL et documentations métier de COPAG, et permet à Vanna de trouver le contexte le plus pertinent par similarité sémantique avant chaque génération.

Streamlit

Streamlit est un framework Python qui transforme des scripts Python en applications web interactives sans nécessiter de développement frontend classique. L'interface COPAG SQL Assistant a été construite avec Streamlit, enrichie de Tailwind CSS injecté dynamiquement pour un rendu visuel professionnel.

4.3 Conclusion

L'ensemble des outils retenus forme un écosystème cohérent, majoritairement open-source. SSIS assure la robustesse des pipelines de données, Vanna.ai et Groq la qualité de la génération SQL, et Streamlit/Plotly l'accessibilité de l'interface. Le chapitre suivant décrit concrètement comment ces outils ont été mis en œuvre.

Chapitre 5

La Réalisation du Projet

4.4 Introduction

Ce chapitre couvre l'ensemble des étapes de développement, dans leur ordre logique d'exécution : création des bases de données, extraction et chargement vers le Data Lake, automatisation et synchronisation via CDC, migration vers le Data Warehouse, intégration de Vanna.ai, entraînement du modèle, et enfin déploiement de l'interface utilisateur.

4.5 Création et Gestion des Bases de Données

La première étape a consisté à créer deux bases de données distinctes dans SQL Server :

1. **La base Data Lake (COPAG_DataLake)** : reçoit les données brutes telles qu'extraites des systèmes sources. Le schéma reprend fidèlement la structure des tables sources sans transformation.
2. **La base Data Warehouse (COPAG_DWH)** : implémente le schéma en flocon défini au chapitre 3, avec les tables de dimensions et la table de faits.

4.6 Extraction et Chargement des Données Sources vers le Data Lake

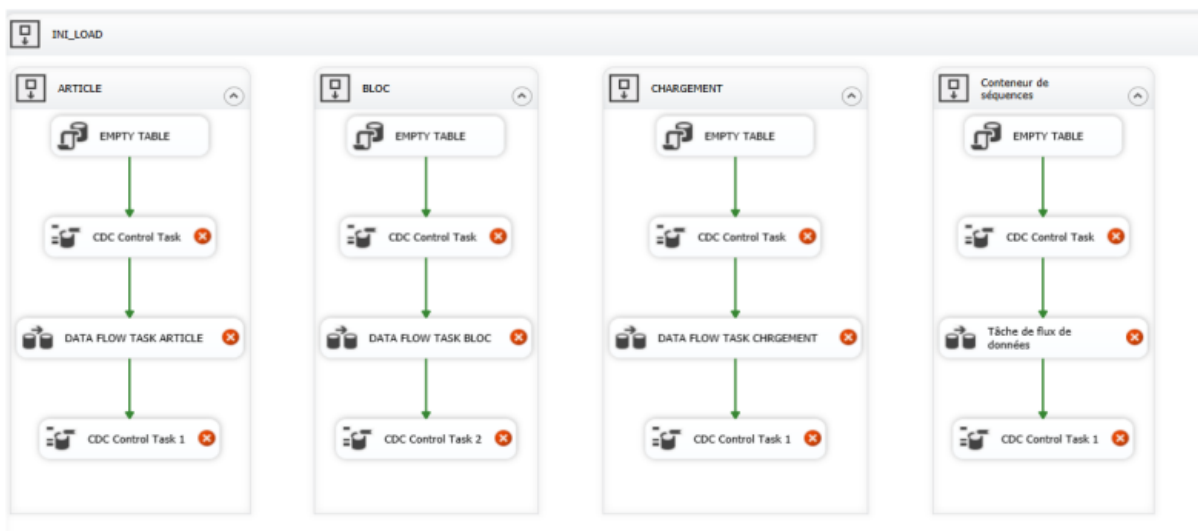


FIGURE 4.1 – Pipeline SSIS — Migration des données sources vers le Data Lake

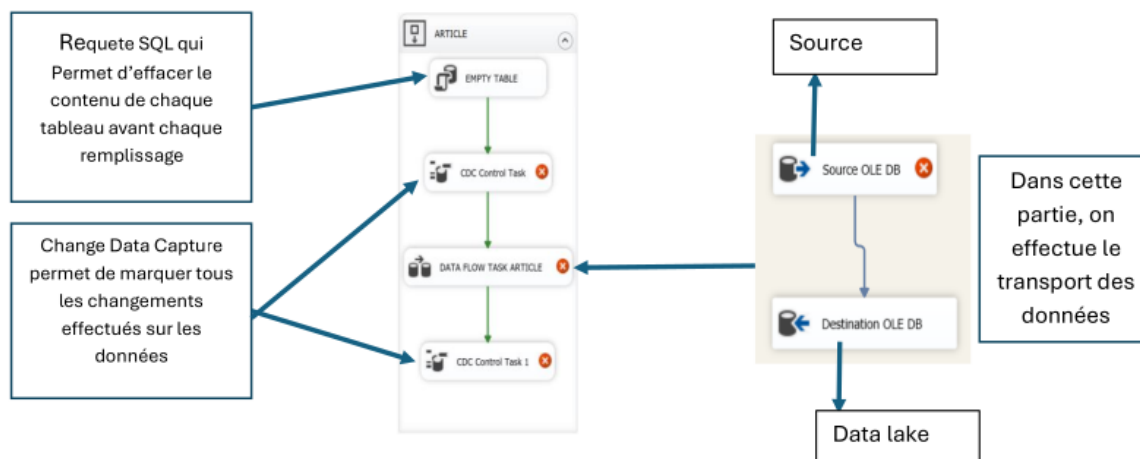


FIGURE 4.2 – Processus détaillé de migration source → Data Lake

Le package SSIS de migration initiale (INI_LOAD) traite en parallèle plusieurs tables sources : ARTICLE, BLOC, CHARGEMENT, etc. Pour chaque table, la séquence est :

1. **EMPTY TABLE** : truncation préalable pour éviter les doublons lors d'un rechargement complet.
2. **CDC Control Task** : marquage du point de départ pour le suivi CDC.
3. **DATA FLOW TASK** : transfert des données via connexions OLE DB source → destination.
4. **CDC Control Task 1** : marquage du point d'arrivée pour les prochaines exécutions incrémentielles.

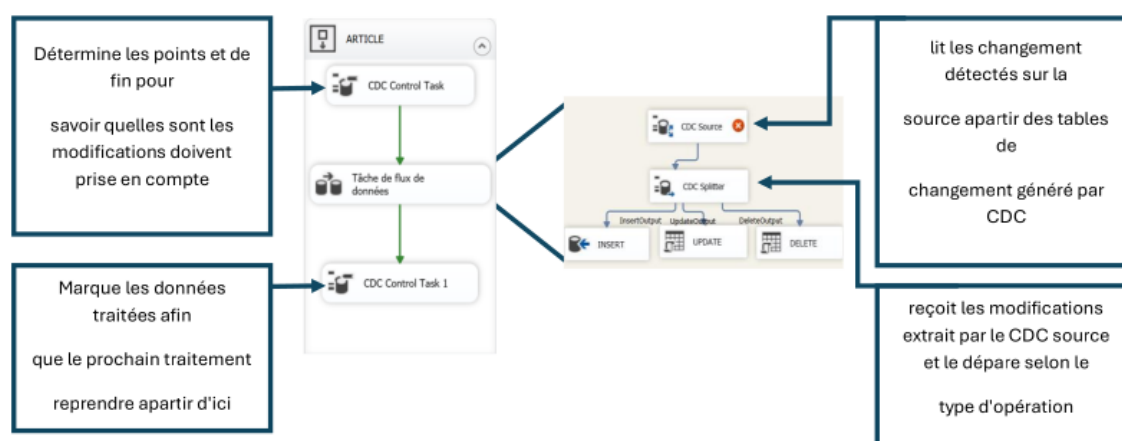


FIGURE 4.3 – Processus d'automatisation — Détail du flux de données article (CDC Source et CDC Splitter)

Le composant **CDC Source** lit les changements détectés dans les tables de la source, tandis que le **CDC Splitter** route les lignes selon leur nature : INSERT, UPDATE ou DELETE.

4.7 Automatisation et Synchronisation

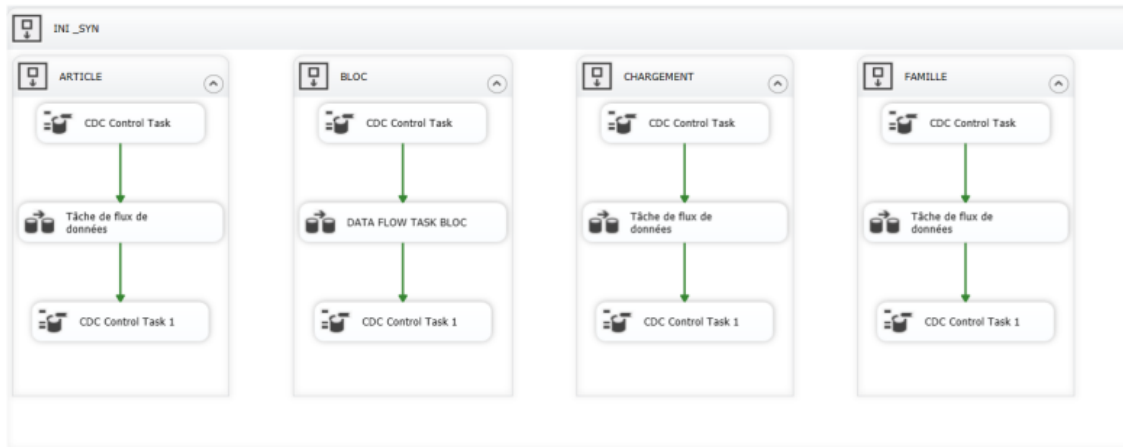


FIGURE 4.4 – Package SSIS d’automatisation et synchronisation incrémentale (INI_SYN)

Une fois le chargement initial réalisé, le package INI_SYN prend le relais pour les synchronisations incrémentielles. Son rôle est de détecter uniquement les enregistrements modifiés depuis la dernière exécution, grâce au mécanisme **CDC** :

- **CDC Control Task (début)** : détermine la plage de LSN (*Log Sequence Number*) à traiter, c’est-à-dire les modifications enregistrées depuis le dernier passage.
- **Data Flow Task** : transporte uniquement les lignes modifiées.
- **CDC Control Task (fin)** : marque les données traitées afin que le prochain passage reparte exactement d’où celui-ci s’est arrêté.

4.8 Migration et Transformation des Données

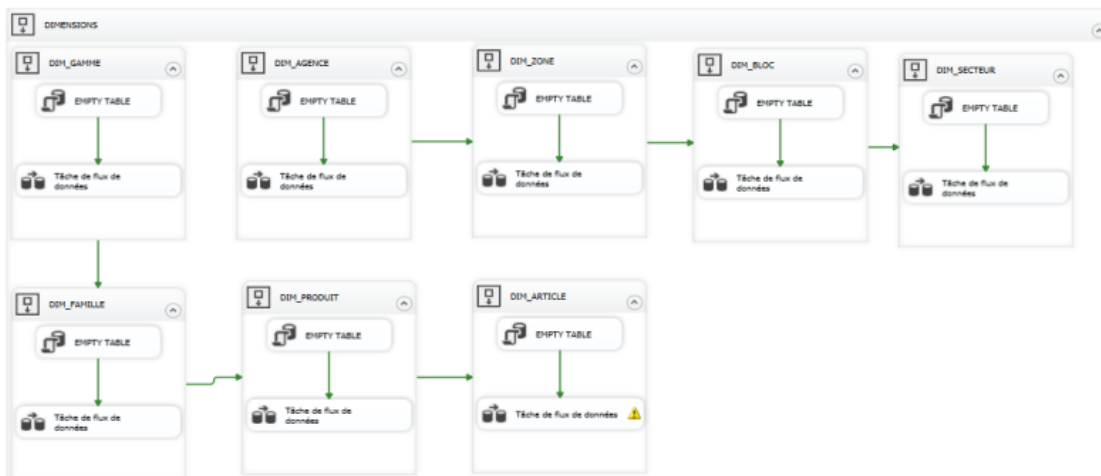


FIGURE 4.5 – Pipeline SSIS — Migration Data Lake vers Data Warehouse (dimensions)

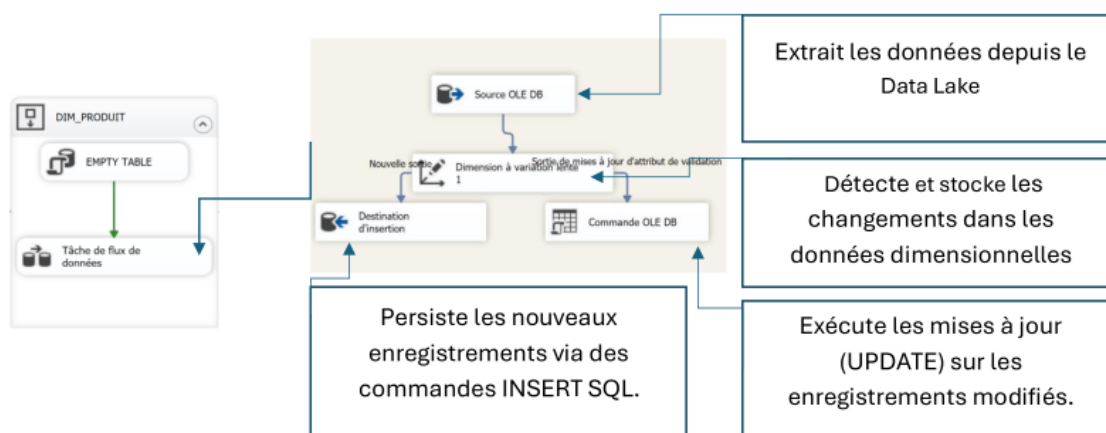


FIGURE 4.6 – Processus de chargement Data Lake → Data Warehouse (DIM_PRODUIT, SCD Type 1)

La migration vers le Data Warehouse implémente une **Slowly Changing Dimension (SCD) de Type 1** pour les tables dimensionnelles. Le composant SSIS utilisé (*Dimension à variation lente*) réalise :

- **Nouveaux enregistrements** → acheminés vers la *Destination d'insertion* pour un INSERT SQL.
- **Enregistrements modifiés** → acheminés vers la *Commande OLE DB* pour un UPDATE SQL.

Ce mécanisme garantit que le Data Warehouse reflète toujours l'état le plus récent des dimensions, sans conserver l'historique des anciennes valeurs (SCD1).

Le package SSIS des dimensions (DIMENSIONS) gère huit tables en séquence : DIM_GAMME, DIM_AGENCE, DIM_ZONE, DIM_BLOC, DIM_SECTEUR, DIM_FAMILLE, DIM_PRODUIT et DIM_ARTICLE, dans un ordre respectant les dépendances de clés étrangères.

4.9 Entraînement de Vanna avec 35 Exemples et Schéma du Data Warehouse

L'entraînement de Vanna a été réalisé en trois étapes :

1. **Entraînement DDL** : les neuf définitions de tables (CREATE TABLE) ont été injectées dans ChromaDB, donnant à Vanna une connaissance complète du schéma.
2. **Documentation métier** : dix règles métier ont été encodées en langage naturel, notamment :
 - la définition du chiffre d'affaires (`qty_unite * prix_vente`),
 - la hiérarchie produit (GAMME → FAMILLE → SOUS_FAMILLE → ARTICLE),

- la hiérarchie géographique (AGENCE → SECTEUR),
- la signification des champs (`qty_rendu`, `qty_invendu`, `qty_promo`).

3. **35 exemples question/SQL** couvrant six blocs analytiques : ventes globales, analyses produit, géographie, clients, promotions/retours, et analyses croisées avancées.

TABLE 4.2 – Exemples de questions entraînées sur Vanna (extrait)

Catégorie	Question
Ventes globales	Quel est le chiffre d'affaires total par année ?
Produits	Quels sont les 10 articles les plus vendus en quantité ?
Géographie	Quel est le chiffre d'affaires par région ?
Clients	Quel est le chiffre d'affaires par canal de distribution ?
Retours	Quels articles ont un taux de retour supérieur à 10% ?
Croisé	Part de marché de chaque gamme sur le CA total ?

4.10 Création de l'Interface et Tests

4.10.1 Partie des Questions

L'interface principale de l'assistant (`frontend/app.py`) propose deux états :

État initial — une page d'accueil centrée avec une barre de recherche proéminente et trois suggestions de requêtes pré-formulées. L'utilisateur tape sa question en français et clique sur « Analyser ».

État résultats — après exécution, l'interface affiche :

- des **cartes KPI** (lignes extraites, total de la première colonne numérique),
- un **bloc AI Insight** : une analyse textuelle de 3-4 phrases générée par le LLM,
- le **tableau de données** complet et interactif (filtrable, triable),
- le **code SQL généré** dans un expandable.

Un **panneau de filtres globaux** dans la barre latérale permet d'affiner les résultats par Agence, Secteur, Gamme, Famille, Article, Année, Mois ou Canal client. Ces filtres sont appliqués à la fois dans la clause WHERE du SQL et en post-filtrage Pandas sur le DataFrame.

4.10.2 Partie de Visualisation



FIGURE 4.7 – Page Principal

La page d'accueil accueille l'utilisateur avec une barre de recherche centrale et une invitation à interroger la base en langage naturel. Des suggestions de requêtes fréquentes sont proposées sous forme de chips cliquables pour guider les utilisateurs moins familiers avec les données. La sidebar affiche les filtres globaux (secteur, agence, gamme, etc.).

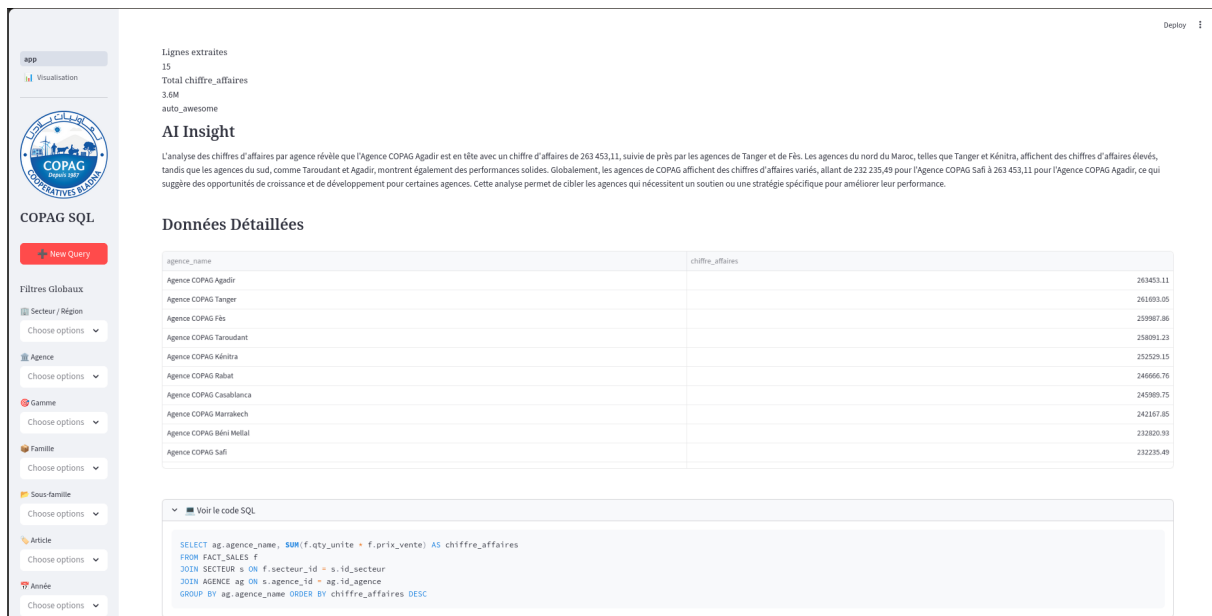


FIGURE 4.8 – Page de résultats — Après soumission d'une requête

Une fois la question soumise, cette page présente les résultats structurés en trois blocs distincts : des métriques synthétiques en haut (nombre de lignes extraites, total calculé),

un encadré "AI Insight" généré automatiquement par le LLM pour interpréter les données en langage métier, et enfin un tableau détaillé des résultats SQL. Le code SQL généré est accessible via un accordéon en bas de page pour les utilisateurs souhaitant vérifier ou réutiliser la requête.

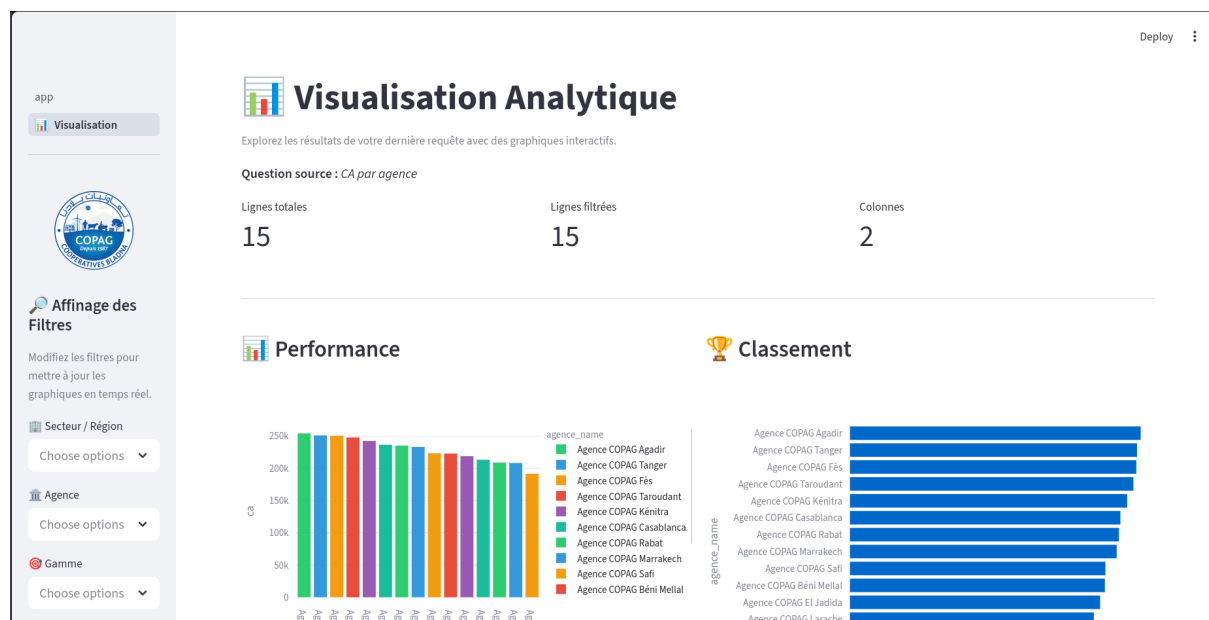


FIGURE 4.9 – Page Visualisation Analytique

Cette page exploite les données de la dernière requête pour produire des graphiques interactifs via Plotly. Elle détecte automatiquement le type de visualisation le plus adapté selon le contenu (barres, classement horizontal, donut...). L'utilisateur peut affiner les graphiques en temps réel grâce aux filtres de la sidebar, sans relancer de nouvelle requête SQL.

4.11 Conclusion

Ce chapitre a décrit l'ensemble des composants développés. L'architecture finale réalise un pipeline complet : les données commerciales de COPAG, extraites de leurs sources, traversent le Data Lake, sont structurées dans le Data Warehouse, et deviennent interrogeables en langage naturel via l'assistant Text-to-SQL. Le résultat est une plateforme décisionnelle accessible à tous les profils, sans prérequis technique.

Conclusion Générale

Conclusion Générale

Ce stage de fin d'études, réalisé au sein de COPAG, a abouti à la conception et au développement d'un système décisionnel complet — de la collecte des données brutes jusqu'à leur consultation en langage naturel.

Sur le plan technique, le projet a démontré la faisabilité d'une architecture moderne combinant un Data Lake SQL Server, un Data Warehouse en schéma flocon, des pipelines SSIS avec CDC, et une couche applicative Text-to-SQL propulsée par Vanna.ai et Groq. L'entraînement sur 35 exemples COPAG et la documentation du domaine métier a permis d'obtenir une génération SQL fiable sur les cas d'usage analytiques les plus fréquents.

Sur le plan métier, la plateforme répond directement au problème initial : un responsable commercial ou un directeur régional peut désormais obtenir une analyse des ventes par gamme, par secteur ou par canal, en moins de cinq secondes, sans dépendre d'une équipe technique.

Ce projet a consolidé des compétences transversales rarement mobilisées simultanément dans un contexte académique : modélisation décisionnelle, développement ETL, ingénierie des LLM, et conception d'interfaces. Il constitue une expérience concrète et complète dans le domaine de la data science appliquée.

Perspectives d'évolution : les prochaines étapes naturelles seraient la connexion directe à SQL Server (en remplacement de SQLite), le déploiement sur serveur interne COPAG, l'extension du corpus d'entraînement à plus de 100 exemples, et l'ajout d'un module de génération de rapports PDF automatiques.

Webographie

1. **Vanna.ai — Documentation officielle**
<https://vanna.ai/docs/>
Framework Text-to-SQL avec RAG.
2. **Groq — Documentation API**
<https://console.groq.com/docs/>
LLM Llama 3 via interface compatible OpenAI.
3. **ChromaDB — Documentation**
<https://docs.trychroma.com/>
Base de données vectorielle open-source.
4. **Streamlit — Documentation**
<https://docs.streamlit.io/>
Framework d'interface web Python.
5. **Plotly — Documentation Python**
<https://plotly.com/python/>
Bibliothèque de visualisation interactive.
6. **Microsoft SSIS — Documentation**
<https://learn.microsoft.com/fr-fr/sql/integration-services/>
SQL Server Integration Services — ETL Microsoft.
7. **COPAG — Site officiel**
<https://www.copag.ma/>
Présentation de la coopérative et de ses activités.
8. **Kimball Group — The Data Warehouse Toolkit**
<https://www.kimballgroup.com/>
Référence pour la modélisation en étoile et en flocon.
9. **Tailwind CSS**
<https://tailwindcss.com/docs/>
Framework CSS utilitaire utilisé pour le design de l'interface.